



Fakultät für Maschinenbau und Mechatronik

Studiengang Robotik und KI i.d. Produktion

Projektbericht

Brick by Brick Foundationlayer: Entwicklung einer bildbasierten Robotersteuerung in AICA

Vorgelegt von: [Bernhard Mohr, Lukas Norz, Magnus Neumann]
Matrikelnummer: [103560, 72779, 103121]

Betreuer (HKA): Prof. Dr.-Ing. habil. Björn Hein

Karlsruhe, den 30. April 2026

Eigenständigkeitserklärung

Wir versichern hiermit, dass wir die vorliegende Projektarbeit selbstständig mit den zugelassenen Hilfsmitteln erstellt haben und diese in der folgenden Erklärung aufführen und kenntlich machen. Alle Teile, die verändert oder unverändert von anderen übernommen wurden, sind als solche kenntlich gemacht und im Literaturverzeichnis vermerkt.

Diese Arbeit wurde mit Unterstützung von KI-Werkzeugen erstellt. Hiermit erklären wir den Einsatz von generativen Text- und Code-KI-Tools. Durch diese kollektive Erklärung am Anfang der Arbeit sind das Dokument und die folgend aufgeführten Punkte als gekennzeichnet zu betrachten. Auf eine kleinteilige Kennzeichnung einzelner Sätze im Haupttext wurde zugunsten des Leseflusses verzichtet.

Im Folgenden wird der Einsatz im Detail erläutert:

Verwendete Werkzeuge: Gemini (Google), Claude (Anthropic) und ChatGPT (OpenAI).

- **Formatierung und Struktur (global):** Die Werkzeuge wurden vollumfänglich zur Unterstützung der LaTeX-Formatierung, der Erstellung von Tabellenstrukturen sowie zur Optimierung von Quellcode-Darstellungen eingesetzt.
- **Sprachliche Überarbeitung (global):** Im gesamten Dokument wurden die Tools unterstützend für die Rechtschreibprüfung, Grammatikkorrektur und teilweise für Formulierungsvorschläge eingesetzt.
- **Formulierungshilfe:** Die folgenden Abschnitte wurden unter Einbezug von KI-basierten Formulierungsvorschlägen erarbeitet und im Anschluss vom Projektteam fachlich geprüft und verantwortet:
 - Kapitel 3, die technischen Beschreibungen der Komponenten-Schnittstellen (Inputs/Outputs).
 - Die Eigenständigkeitserklärung.
 - Die Kurzfassung in deutscher Sprache sowie der Abstract in englischer Sprache.
 - Die Zusammenfassung und das Fazit.

Karlsruhe, den 30. April 2026

Unterschriften:



Magnus Neumann



Bernhard Mohr



Lukas Norz

Kurzfassung

Brick by Brick Foundationlayer: Entwicklung einer bildbasierten Robotersteuerung in AICA

Die vorliegende Projektarbeit befasst sich mit der Entwicklung einer flexiblen, bildbasierten Robotersteuerung unter Nutzung der Plattform AICA und ROS 2. Ziel des Projekts „Brick by Brick“ ist es, ungeordnet im Arbeitsraum liegende Jenga-Bausteine mithilfe einer am Roboterarm (KUKA) montierten Intel RealSense-Kamera zu detektieren und diese autonom entlang einer handgezeichneten Linie auf einem Blatt Papier zu platzieren.

Für die visuelle Wahrnehmung wurde eine zweistufige Pipeline entwickelt: Die Bausteinererkennung erfolgt echtzeitfähig über das Instanzsegmentierungsmodell YOLOv11 (Nano), während die Linienextraktion auf dem Papier durch einen maßgeschneiderten Computer-Vision-Algorithmus realisiert wird. Die Systemarchitektur gliedert sich in eine Explorationsphase zur Arbeitsraumerfassung und eine anschließende Pick-and-Place-Phase. Zur Reduktion von Race-Conditions und der CPU-Last wurde die Software in einer zweiten Architekturvariante zu einem zentralen „Vision Processor“ konsolidiert.

Die Ergebnisse zeigen, dass die KI-gestützte Wahrnehmung und Pfadplanung in der Simulation zuverlässig funktionieren. In der realen Umsetzung zeigten sich jedoch technische Limitierungen: Eine hohe CPU-Auslastung führte zu Systeminstabilitäten, und die fehlende I/O-Schnittstellenanbindung zwischen AICA und der KUKA-Steuerung verhinderte die physische Ansteuerung des Vakuumgreifers.

Abstract

Brick by Brick Foundationlayer: Entwicklung einer bildbasierten Robotersteuerung in AICA

This project report details the development of a flexible, image-based robot control system using the AICA platform and ROS 2. The goal of the "Brick by Brick" project is to detect randomly scattered Jenga blocks using an Intel RealSense camera mounted on a KUKA robot arm, and to autonomously place them along a hand-drawn line on a piece of paper.

For visual perception, a two-stage pipeline was developed: Real-time block detection is achieved using the YOLOv11 (Nano) instance segmentation model, while the extraction of the line paths is handled by a custom computer vision algorithm. The system architecture is divided into an exploration phase for workspace mapping and a subsequent pick-and-place phase. To reduce race conditions and CPU load, the system components were consolidated into a central "Vision Processor" in a second architectural iteration.

The results demonstrate that the AI-supported perception and path planning function reliably in simulation. However, the physical implementation faced technical limitations: High CPU load caused system instabilities, and the lack of I/O interface connectivity between AICA and the KUKA controller prevented the physical operation of the vacuum gripper.

Inhaltsverzeichnis

Abkürzungsverzeichnis	VI
1 Einführung	1
1.1 Motivation und Stand der Technik	1
1.2 Aufgabenstellung und Rahmenbedingungen	2
1.3 Grundlagen: Docker, ROS 2 und AICA	2
2 Angewandte Bildverarbeitung	4
2.1 RGB-Bildererkennung von Bausteinen mit YOLOv11	4
2.1.1 Anwendungsumstände und Modellwahl	4
2.1.2 Datensatz und Training	4
2.1.3 Ergebnisse und Modellwahl	6
2.2 Computer Vision	7
2.2.1 Ablauf des Programms	7
2.2.2 Pipeline Grundlinie	8
2.2.3 Blatterkennung (Papermask)	8
2.2.4 Linienextrahierung	9
2.2.5 Blockplatzierung und Transformation	11
3 Systemablauf - Komponentenbeschreibungen	12
3.1 Lösungskonzept und Systemablauf	12
3.2 Exploration Navigator	13
3.3 JTC Command Generator	15
3.4 Pose Triggered Camera	16
3.5 YOLO Object Detector	17
3.6 Dropoff Line Extractor	18
3.7 Master List Manager	19
3.8 Pick Place Controller	20
3.9 Architektur 2: Vision Processor Komponente	22
4 Probleme und Schwierigkeiten	25
4.1 Evaluierung und konstruktives Feedback zur AICA-Plattform	25
5 Zusammenfassung und Fazit	27
5.1 Zusammenfassung	27
5.2 Fazit	27
Tabellenverzeichnis	28
Abbildungsverzeichnis	28
Literaturverzeichnis	28

Abkürzungsverzeichnis

Abkürzungen¹

HKA	Hochschule Karlsruhe
JTC	Joint Trajectory Controller
MLM	Master List Manager
PPC	Pick Place Controller
VP	Vision Processor

¹ Abkürzungen, die bereits im Duden stehen, werden nicht aufgeführt.

1 Einführung

1.1 Motivation und Stand der Technik

Flexible und adaptiv einsetzbare Robotersysteme gewinnen in der industriellen Automatisierung zunehmend an Bedeutung, da sich Produktionsumgebungen immer häufiger durch wechselnde Bauteile, unstrukturierte Arbeitsräume und steigende Anforderungen an die Wiederverwendbarkeit von Anwendungen auszeichnen. In der klassischen Industrierobotik sind Anwendungen in der Regel stark an einen spezifischen Robotertyp gebunden. Hersteller wie KUKA, ABB oder Fanuc verwenden proprietäre Programmiersprachen und Schnittstellen, wodurch identische Anwendungen für unterschiedliche Systeme oft vollständig neu implementiert werden müssen. Diese fehlende Standardisierung stellt eine enorme Hürde dar, da sowohl die Kommunikation mit der Hardware als auch die Bewegungssteuerung individuell angepasst werden müssen und somit einen hohen Entwicklungsaufwand verursachen.

Moderne Ansätze wie ROS 2 versuchen dieses Problem durch eine modulare und standardisierte Kommunikationsstruktur zu adressieren, indem sie eine Abstraktion zwischen Softwarekomponenten und Hardware schaffen. Dennoch bleibt die Integration realer Industrieroboter komplex, da herstellerspezifische Treiber und Schnittstellen weiterhin erforderlich sind und eine durchgängige, echtzeitfähige Steuerung oft schwer zu realisieren ist.

An dieser Stelle setzt das Framework AICA an, welches eine zusätzliche Abstraktionsebene über ROS 2 bereitstellt und speziell auf industrielle Anwendungen ausgelegt ist. AICA kombiniert standardisierte Softwaremodule mit hardwareunabhängigen Schnittstellen und ermöglicht es, Roboter verschiedener Hersteller über eine einheitliche Steuerungslogik anzusprechen. Anwendungen können dabei aus wiederverwendbaren Bausteinen zusammengesetzt und mithilfe einer graphbasierten Umgebung entwickelt werden. Dies führt zu der Möglichkeit, einmal entwickelte Anwendungen mit geringem Anpassungsaufwand auf andere Robotersysteme zu übertragen, wodurch die sonst übliche enge Kopplung zwischen Software und Hardware weitgehend aufgelöst wird.

Darüber hinaus erlaubt AICA durch seine Echtzeitfähigkeit und die Integration sensorbasierter Regelungen eine adaptive Steuerung, bei der Roboter auf Veränderungen in ihrer Umgebung reagieren können. Während klassische Automatisierungssysteme häufig auf starre, vorprogrammierte Abläufe beschränkt sind, ermöglicht dieser Ansatz eine deutlich höhere Flexibilität und Robustheit.

Vor diesem Hintergrund wird im Folgenden die konkrete Aufgabenstellung und die Rahmenbedingungen des Projekts beschrieben.

1.2 Aufgabenstellung und Rahmenbedingungen

Im Rahmen des vorliegenden Projekts soll ein robotergestütztes System entwickelt werden, das auf Basis von AICA in der Lage ist, auf einem DIN-A4-Blatt Papier gezeichnete Linien zu erkennen und entlang dieser Jenga-Bausteine abzulegen. Die Bausteine befinden sich dabei zu Beginn ungeordnet im Arbeitsraum des Roboters und müssen zunächst detektiert sowie in ihrer Lage und Ausrichtung bestimmt werden, bevor eine strukturierte Platzierung erfolgen kann. Ziel ist es, die erkannte Linie möglichst vollständig und sinnvoll mit den vorhandenen Bausteinen zu belegen.

Die Form der Linie, sowie Position und Ausrichtung des Blattes sind dabei nicht vorgegeben und können beliebig variieren, wodurch eine robuste und allgemein anwendbare Lösung erforderlich ist. .

Die Umsetzung erfolgt innerhalb einer bestehenden Roboterzelle mit einem Kuka KR 3 540 Roboterarm. Zur Erfassung der Umgebung wird eine am Roboter montierte Intel Kamera RealSense435i verwendet, die gemeinsam mit dem Manipulator bewegt wird. Der Greifprozess wird mittels eines Vakuumgreifers realisiert. Die Steuerung und Koordination der einzelnen Systemkomponenten erfolgt über das Framework AICA in Kombination mit ROS 2. Der Prozess soll bis auf die Initialisierung vollständig autonom ablaufen und ist abgeschlossen wenn die Linie vollständig mit Bausteinen belegt ist.

Der Fokus des Projekts liegt somit nicht auf einer fest vorgegebenen Lösung, sondern auf der Entwicklung eines Systems, das unter den gegebenen Randbedingungen flexibel auf unterschiedliche Szenarien reagieren kann und eine übertragbare, modulare Struktur aufweist.

1.3 Grundlagen: Docker, ROS 2 und AICA

Docker dient der Simulation einer Betriebsumgebung inklusive aller benötigten Programme (Python/C++) und Pakete, vergleichbar mit einer virtuellen Maschine ohne grafische Oberfläche. Ein solches ausgeführtes System wird als **Container** bezeichnet, während sämtliche Konfigurationsdaten und Abhängigkeiten in einem statischen **Image** hinterlegt sind. Durch die Verwendung von Images lässt sich eine exakt definierte Arbeitsumgebung reproduzieren, was eine systemunabhängige Konsistenz der Programme garantiert, da diese innerhalb ihrer isolierten Containerumgebung operieren.

- **Docker-Befehle im Terminal:** `docker image ls` (Anzeigen der verfügbaren Images), `docker image rm [ID]` (Löschen eines bestimmten Images), `docker system prune` (System von ungenutzten Ressourcen bereinigen).

ROS 2 (Robot Operating System 2) ist ein Open-Source-Framework für den Betrieb und die Steuerung von Robotern in einem Netzwerk, welches bei Bedarf durch eine hohe Taktfrequenz echtzeitfähig ist. Das Grundkonzept basiert auf **Nodes**, die als modulare Steuereinheiten fungieren. Jeder Node verfügt über Ein- und Ausgänge sowie ein zugehöriges Skript (z. B. `.py`). Die Kommunikation erfolgt primär über **Topics**, auf denen Informationen im Takt des ROS-Netzwerks veröffentlicht bzw. aktualisiert (Publish) und von anderen Teilnehmern abgegriffen (Subscribe) werden können. Während Topics für den kontinuierlichen Datenfluss zuständig

sind, dienen **Services** (Client-Service-Modell) dem asynchronen Aufrufen spezifischer Funktionen außerhalb der festen Netzwerkfrequenz.

- **Wichtige ROS 2 Terminal-Befehle:** `ros2 topic list` (Übersicht der Topics im Netzwerk), `ros2 service list` (Übersicht der verfügbaren Services), `ros2 topic echo [Name]` (Datenstrom eines Topics auslesen).

AICA nutzt ROS 2 als Basis und erweitert es um eine grafische Benutzeroberfläche sowie spezialisierte Werkzeuge. In AICA Studio wird das ROS-Netzwerk als Blockschaltbild visualisiert, wobei Topics als Verbindungslinien zwischen den Blöcken dargestellt werden. Besondere Merkmale sind die herstellerunabhängige Steuerung sowie die Echtzeitfähigkeit. Über den **TF-Tree** werden alle Koordinatensysteme (*Frames*) der Roboterzelle verwaltet. Die Steuerung erfolgt über verschiedene Controller, wie den **Joint Trajectory Controller** (JTC) für geplante Bewegungen oder die **Inverse Kinematics** (IK) für kartesische Zielvorgaben. AICA lässt sich durch ein auf GitHub bereitgestelltes Software Development Kit (SDK) um eigene Komponenten (Blöcke) erweitern.

- **Wichtiger AICA-SDK-Befehl:** `docker build -f ./aica-package.toml -t [Name]`
`.` (Eigenes Paket als Image bauen).

2 Angewandte Bildverarbeitung

2.1 RGB-Bilderkennung von Bausteinen mit YOLOv11

2.1.1 Anwendungsumstände und Modellwahl

Die Grundlage der Bildverarbeitung bildet der RGB-Stream der auf dem Roboter montierten Intel RealSense-Kamera. Aufgrund des Einsatzes in einem hochfrequenten Echtzeitsystem muss die Bildverarbeitung sowohl schnell als auch zuverlässig erfolgen. Hinsichtlich ihrer Form und Farbe weisen die zu erkennenden Bausteine eine verhältnismäßig geringe Komplexität auf.

Um hierbei eine präzise und zugleich flexible Erkennung zu gewährleisten, kommt das Verfahren der Instanzsegmentierung zum Einsatz. Dabei werden aus den Bildinformationen Polygone extrahiert, die anschließend mittels OpenCV (über die Funktionen `minAreaRect` und `boxPoints`) zu orientierten Boundingboxen (im Folgenden OBB) weiterverarbeitet werden. Diese Repräsentation reicht aus, um die Geometrie der Bausteine für die Erkennung hinreichend abzubilden. Die Beschränkung auf eine zweidimensionale Form resultiert aus dem spezifischen Ablauf des Greifprozesses: Zur exakten Erkennung wird die Kamera orthogonal über den vorab grob lokalisierten Blöcken positioniert, wodurch die Seitenkonturen aufgrund des Blickwinkels nicht mehr sichtbar sind.

Neben der konventionellen Generierung von OBBs bietet die Instanzsegmentierung grundsätzlich die Möglichkeit, den präzisen Informationsgehalt der extrahierten Polygone durch eigene Verfahren weitergehend zu nutzen. Dieses Potenzial stellte zu Projektbeginn ein zusätzliches Auswahlkriterium dar; im finalen Systemstadium findet es jedoch keine Anwendung, da bereits die extrahierten OBBs die erforderliche Genauigkeit erreichen.

Aufgrund der Anforderungen an Genauigkeit und Verarbeitungsgeschwindigkeit fiel die Wahl auf das Modell YOLOv11. Die Entscheidung für ein YOLO-basiertes Segmentierungsmodell begründet sich primär durch dessen architektonische Effizienz. Während klassische, zweistufige Segmentierungsnetzwerke (wie Mask R-CNN) für Hochfrequenzanwendungen in der Regel zu langsam sind, löst das einstufige YOLO-Paradigma dieses Problem durch eine hochgradig parallelisierte Berechnung von Instanzmasken und räumlichen Merkmalen [2, 4]. Diese Effizienz ermöglicht deterministische Latenzen und sehr hohe Bildwiederholraten [6]. YOLOv11 stellt derzeit eine der leistungsfähigsten Lösungen dar, um den komplexen Trade-off zwischen detaillierter Merkmalsextraktion und Echtzeitanforderungen aufzulösen [3].

2.1.2 Datensatz und Training

Für das Training des Modells wurde ein Datensatz bestehend aus 533 Bildern erstellt. Die Aufnahmen umfassen variierende Anzahlen an Bauklötzen, welche teilweise auch in gestapelter Form abgebildet sind (siehe Abb. 2.1). Zusätzlich wurden Bilder ohne Zielobjekte sowie

Aufnahmen mit einem Blatt Papier und eingezeichneter Grundlinie hinzugefügt, um Fehlklassifikationen zu vermeiden.

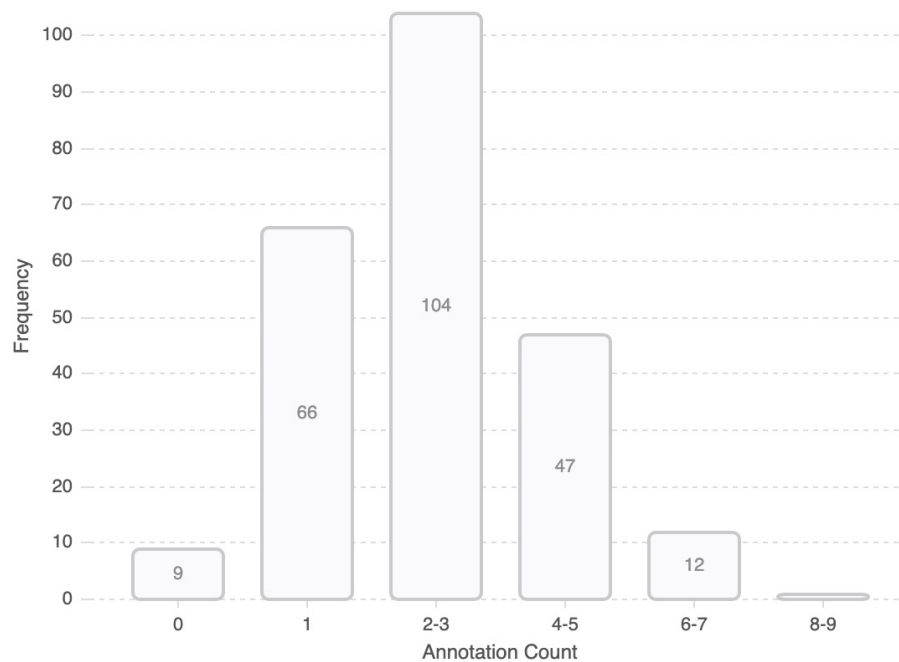


Bild 2.1. Datenverteilung der Bauklötze pro Bild.

Die Bildaufnahme erfolgte unter Realbedingungen über die Intel RealSense-Kamera zu verschiedenen Tageszeiten, um unterschiedliche Beleuchtungsszenarien zu berücksichtigen. Der Datensatz wurde in einem Verhältnis von 70-15-15 in Trainings-, Validierungs- und Testdaten unterteilt. Die Datenannotation erfolgte mithilfe der Plattform Roboflow [1], wobei die Zielobjekte mit Polygonen versehen wurden (siehe Abb. 2.2).

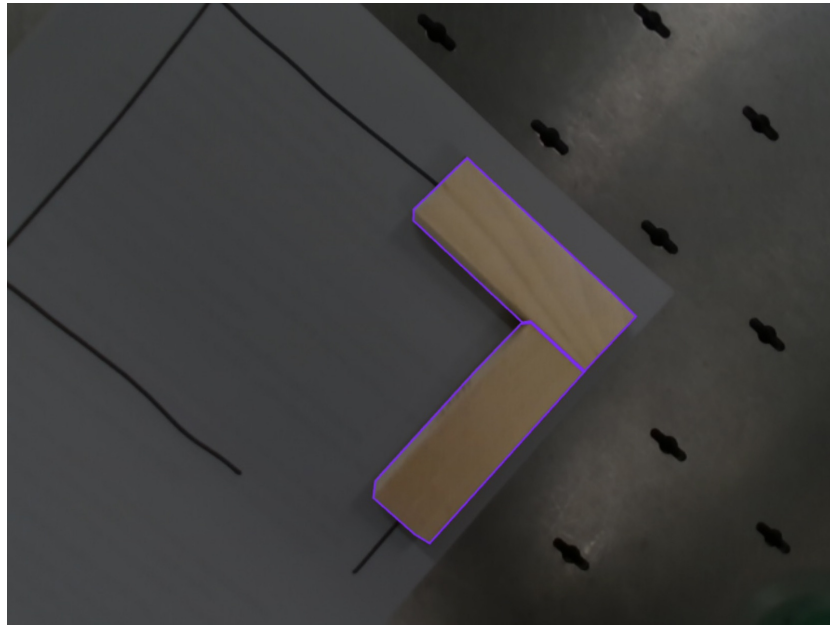


Bild 2.2. Annotiertes Beispielbild in der Plattform Roboflow: Kombination aus Grundlinienblatt und Blöcken.

Innerhalb des Datensatzes wurde die Kategorie „Woodblocks“ definiert. Auf eine explizite Vorverarbeitung des Datensatzes wurde verzichtet. Es wurden mehrere YOLOv11-Modelle (Nano, Small, Medium) trainiert, wobei die Nano-Variante aufgrund ihrer geringen Latenzen und reduzierten Rechenlast bevorzugt wurde [5]. Das Training erfolgte über 80 Epochen unter Einsatz der „Mosaic“-Augmentierung, welche in der Endphase (letzte 10 bis 20 Epochen) zur Steigerung der Präzision deaktiviert wurde.

2.1.3 Ergebnisse und Modellwahl

Die Evaluation der trainierten Modelle anhand der Testbilder lieferte zuverlässige Ergebnisse.

Tabelle 2.1. Ergebnisse der Fehleranalyse und Inferenzzeiten der trainierten Modelle

Model	Images Tested	Misses (False Neg)	Ghosts (False Pos)	Avg Accuracy (IoU)	Avg Confidence	Avg Inference (ms)	Total Benchmark Time (s)
weights_s_optimized_last	79	1	3	0.9575	0.9249	52.89	4.28
weights_s_optimized_best	79	4	2	0.9573	0.9208	50.72	4.11
weights_n_optimized_last	79	2	2	0.9551	0.9344	32.12	2.65
weights_n_optimized_best	79	2	2	0.9551	0.9330	30.02	2.47
weights_m_standard_last	79	5	1	0.9528	0.9119	107.66	8.61
weights_m_standard_best	79	5	1	0.9528	0.9119	106.87	8.54
weights_s_standard_last	79	2	3	0.9494	0.9092	49.85	4.04
weights_s_standard_best	79	3	1	0.9535	0.8960	56.21	4.55

Alle Modelle liefern auf Basis des Datensatzes Ergebnisse mit einer Genauigkeit von über 0,95. Das Nano-Modell erzielte dabei marginal bessere Werte und wies die geringste Anzahl an Falschdetektionen auf (siehe Abb. 2.3).

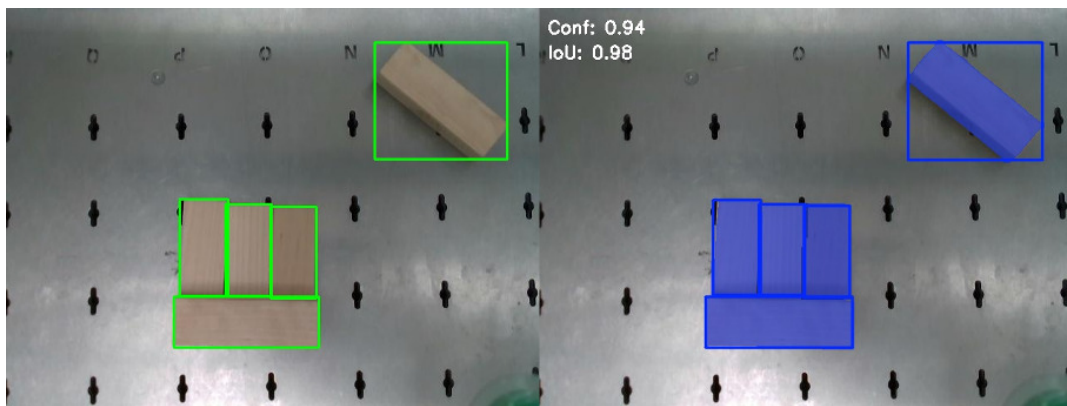


Bild 2.3. Ergebnisse der Modelle bezüglich False-Positiv- und False-Negativ-Detektionen.

Gleichzeitig liefert die Nano-Variante mit einer Benchmarkzeit von unter 2,5 Sekunden die schnellsten Ergebnisse. Die ergänzende visuelle Überprüfung (siehe Abb. 2.4) bestätigt, dass das optimierte Nano-Modell die Anforderungen am besten erfüllt und daher für den weiteren Projektverlauf ausgewählt wurde.

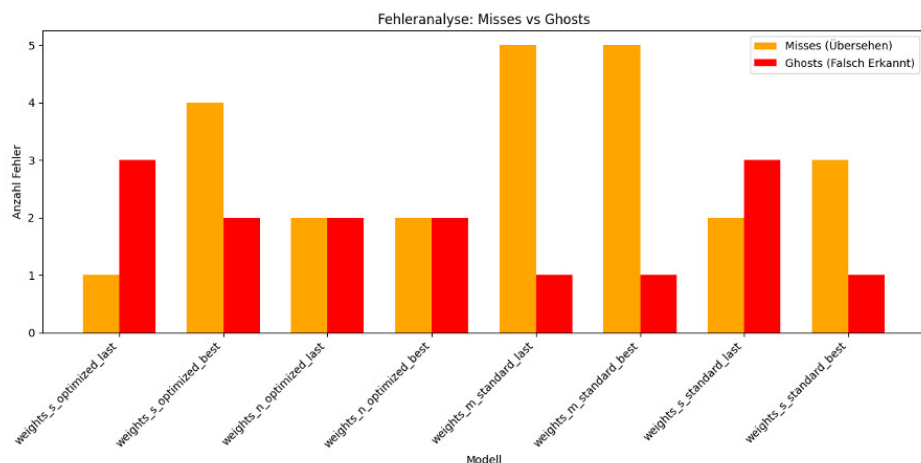


Bild 2.4. Beispiel-Testbild des optimierten YOLOv11-Nano-Modells mit Masken und Boundingboxen.

2.2 Computer Vision

2.2.1 Ablauf des Programms

Eine der Herausforderungen im „Brick-by-Brick“-Projekt liegt in der zuverlässigen Erkennung der Grundlinie, auf der der Roboter die Bausteine ablegen soll. Das System muss so ausgelegt sein, dass variabel geformte Linien auf Papierblättern in unterschiedlichen Positionen, Ausrichtungen und bei wechselnden Lichtverhältnissen stabil erfasst werden. Anschließend sollen diese gefundenen Linien möglichst kompakt mit Bausteinen belegt und deren finale Positionen sowie Orientierungen ausgegeben werden. Aufgrund der stark reflektierenden und kantenreichen Tischoberfläche erweist sich dies als besonders schwierig, da einfache Kantendetektionsver-

fahren hierfür nicht ausreichen. Im Folgenden wird zunächst die Pipeline des Programms im Überblick vorgestellt, bevor die konkrete Umsetzung der einzelnen Schritte erläutert wird.

2.2.2 Pipeline Grundlinie

Als Eingangsdaten dienen ein Kamerabild sowie die dazugehörige Roboterpose zum Zeitpunkt der Aufnahme. Zunächst prüft das Programm, ob ein Blatt Papier im Bild sichtbar ist und bestimmt dessen genaue Umrandung. Innerhalb dieses isolierten Blattbereichs wird anschließend nach den gezeichneten Linien gesucht, welche die gewünschten Grundlinien für die Bausteine darstellen. Sobald diese Linienpfade extrahiert sind, berechnet der Algorithmus eine möglichst kompakte und kollisionsfreie Anordnung von Rechtecken entlang dieser Linien. Nach der Ermittlung der Platzierungspunkte erfolgt eine Transformation von Pixel- in Weltkoordinaten. Als finaler Output wird eine Liste mit den exakten Positionen und Orientierungen der Ablageorte der Bausteine ausgegeben.

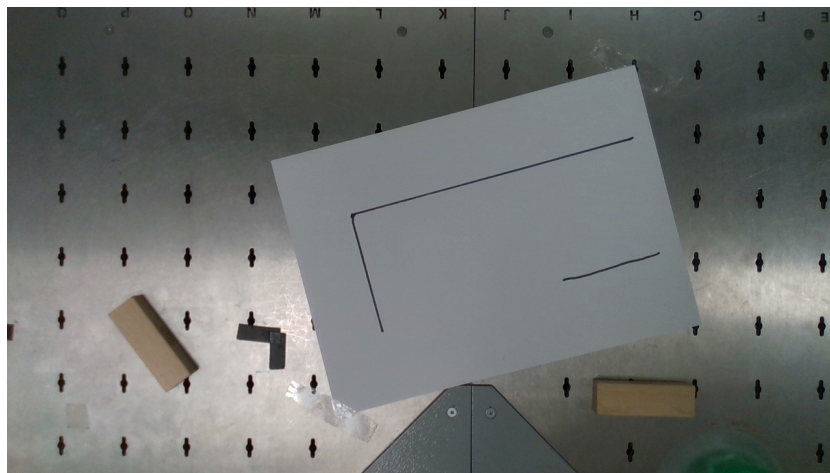


Bild 2.5. Kamera-Eingangsbild der Arbeitsfläche.

2.2.3 Blatterkennung (Papermask)

Die Isolierung des Arbeitsbereichs ist aufgrund der anspruchsvollen Umgebung zwingend erforderlich. Wie in Abbildung 2.5 ersichtlich, führt die metallische, reflektierende Tischoberfläche mit ihrer Lochstruktur bei einfachen Kantendetektionsverfahren zu massiven Artefakten. Da sich das weiße Papier farblich kaum vom hellen Metall abhebt, reicht eine reine Schwellenwertbetrachtung nicht aus. Daher wird eine Pipeline eingesetzt, die verschiedene Bildmerkmale kombiniert (vgl. Abb. 2.6):

Likelihood-Map und Segmentierung Zunächst wird eine Wahrscheinlichkeitskarte (*Likelihood P*) erstellt. Hierbei werden neben der Helligkeit vor allem die Farbsättigung (Chroma) und die lokale Texturhomogenität bewertet. Da das Papier im Gegensatz zum gelochten Metalltisch eine sehr glatte Oberfläche aufweist, lässt es sich so stabil abgrenzen. Diese Karte dient als Basis für ein GrabCut-Verfahren, das den Vordergrund (Papier) iterativ vom Hintergrund (Tisch) trennt.

Maskenoptimierung und Quad-Rekonstruktion Die resultierende Rohmaske wird durch morphologische Operatoren bereinigt, um Rauschen zu unterdrücken und die größte zusammenhängende Fläche zu isolieren (*Largest component*). Um eine robuste Basis für die Robotersteuerung zu erhalten, wird die gefundene Kontur mathematisch in ein Viereck überführt (*Quad overlay*). Dies funktioniert, weil davon ausgegangen werden kann, dass das Blatt ohne Wölbungen auf dem Tisch liegt und so mit einem einfachen Rechteck abgebildet werden kann. Durch das Fitting von Linien an die Maskenränder bleibt die Erkennung auch bei teilweisen Verdeckungen oder Schattenwurf an den Papierkanten stabil.

Resultat Das Ergebnis dieses Prozesses ist ein entzerrter, auf das Wesentliche reduzierter Bildausschnitt (*Warped A4*), der ausschließlich die für die weitere Linienextrahierung relevanten Informationen enthält. Durch dieses manuelle Fine-Tuning der Parameter wird sichergestellt, dass Reflexionen auf dem Metalltisch die nachfolgende Pfadplanung nicht negativ beeinflussen.

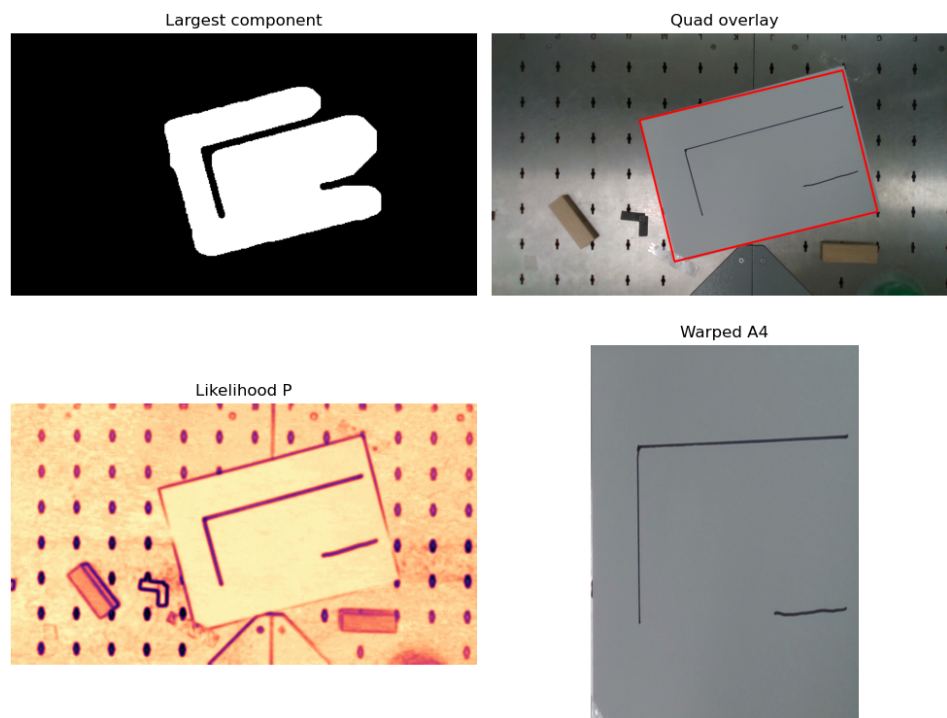


Bild 2.6. Visualisierung der Papermask-Erstellung und Blatt-Segmentierung.

2.2.4 Linienextrahierung

Nachdem das Blatt Papier als isolierte „Region of Interest“ (ROI) vorliegt, konzentriert sich der Algorithmus auf die Erfassung der handgezeichneten Grundlinie. Trotz der bereinigten Umgebung des *Warped A4* Bildes ist die Extraktion einer mathematisch nutzbaren Trajektorie komplex, da die realen Zeichnungen variierende Strichstärken und topologische Unregelmäßigkeiten aufweisen.

Methodische Entwicklung und verworfene Ansätze Im Zuge der Entwicklung werden verschiedene Strategien evaluiert, die sich jedoch als nicht hinreichend robust erweisen. Hough-basierte Verfahren zur Blattkantenerkennung scheitern häufig an den dominanten Metallkanten des Tisches. Frühe Versuche einer Pfadfindung mittels *Root-to-Leaf*-Graphen-Traversierung oder eines lokalen *Edge-Pairings* an Verzweigungen (Junctions) zeigen Defizite bei komplexen Symmetrien: In Bildern mit starken Reflexionen oder lückenhaften Skeletons führen diese Ansätze dazu, dass Segmente unberücksichtigt bleiben oder die Linienführung an Kreuzungspunkten instabil wird.

Preprocessing und Skeletonisierung Die finale Lösung nutzt ein adaptives Preprocessing, um Linien unabhängig von ihrer Strichstärke stabil zu isolieren. Anschließend erfolgt die **Skeletonisierung**. Dieser Prozess reduziert die Linien auf eine Breite von genau einem Pixel, um das Zentrum der Zeichnung zu finden. Ein wesentliches technisches Problem besteht hierbei jedoch darin, dass das Ergebnis lediglich eine Menge unverbundener Pixel liefert, jedoch noch keine durchgehende, mathematisch beschreibbare Trajektorie.

Trajektorienbildung und Gap-Closing Um aus den fragmentierten Pixeldaten eine nutzbare Bahn zu generieren, werden zusätzliche Funktionen zur Graphen-Konstruktion implementiert. Ein **Gap-Closing-Algorithmus** überbrückt automatisch kleine Unterbrechungen in der Zeichnung. Beim anschließenden **Stroke-Building** wird ein Ansatz verfolgt, der alle Pixel einbindet. Anstatt an Abzweigungen fehleranfällige lokale Paarungen vorzunehmen, wird der gesamte Graph so abgearbeitet, dass jedes erkannte Segment in die Pfadplanung einfließt. Dies stellt sicher, dass auch bei komplexen Zeichnungen keine Informationen verloren gehen und eine lückenlose Basis für die Robotersteuerung entsteht.

- **Gap-Closing:** Ein Algorithmus sucht nach Endpunkten von Liniensegmenten, die in geringem Abstand zueinander liegen, und überbrückt diese Lücken automatisch.
- **Stroke-Building:** Die einzelnen Pixel werden in eine geordnete Liste von Koordinaten überführt. Dabei muss das System entscheiden, wie es an Abzweigungen verfährt. Ein Cover-all-Ansatz stellt sicher, dass alle erkannten Segmente in mathematische Pfade umgewandelt werden, ohne dass kurze Liniestücke als Rauschen verworfen werden.

Das Ergebnis (vgl. Abb. 2.7) ist eine saubere Menge von Pfaden, die die geometrische Grundlage für die nachfolgende Blockplatzierung bilden.

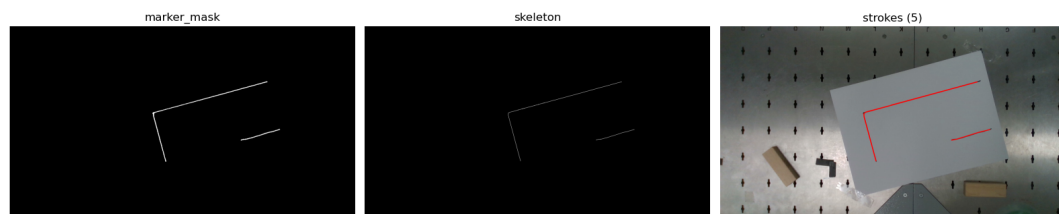


Bild 2.7. Darstellung der extrahierten Skelett-Linien.

2.2.5 Blockplatzierung und Transformation

Nach der erfolgreichen Extraktion der Linienpfade folgt die Überführung dieser geometrischen Daten in konkrete Handlungsanweisungen für den Roboter. Dies umfasst die algorithmische Anordnung der Bausteine sowie die Umrechnung der Bilddaten in das reale Weltkoordinatensystem.

Platzierungsalgorithmus und Kollisionsprüfung Um die extrahierten Linien effizient zu nutzen, berechnet ein Platzierungsalgorithmus die optimalen Positionen für die Holzbausteine (vgl. Abb. 2.8). Ziel ist eine möglichst kompakte Belegung entlang der Pfade, wobei die physischen Dimensionen der Blöcke als Rechtecke modelliert werden. Während des Prozesses wird kontinuierlich eine Kollisionsprüfung durchgeführt: Neue Blöcke werden nur dann platziert, wenn sie weder bestehende Bausteine überlagern noch die Blockmittelpunkte über die Grenzen der erkannten Linien hinausragen. Zur Optimierung der Roboterbewegung wird zudem der Gierwinkel (Yaw) so angepasst, dass die Symmetrie der Steine ausgenutzt wird, was unnötige Achsrotationen des Greifers minimiert.

Transformation in Weltkoordinaten Ein entscheidender Vorteil des Systemaufbaus liegt in der kontrollierten Kamerapose während der Aufnahme. Da die Kamera bei der Aufnahme des Blattes stets exakt orthogonal zum Blatt Papier ausgerichtet ist und lediglich Rotationen um die Z-Achse (Yaw) vollzieht, vereinfacht sich die mathematische Projektion erheblich. Da zudem der vertikale Abstand zum Tisch konstant und bekannt ist, kann aus der aktuellen Roboterpose direkt auf die Weltkoordinaten des Bildmittelpunkts sowie aller weiteren Pixel geschlossen werden.

Ein manueller Ausgleich von Objektivverzerrungen ist hierbei nicht erforderlich, da die hardwareseitige Korrektur der Kamera bereits durch den Hersteller erfolgt und somit geometrisch korrekte Bilddaten geliefert werden. Die Transformation nutzt eine homogene Matrix, um die Pixelkoordinaten unter Berücksichtigung der Kamerapose (die in den Weltkoordinaten bekannt ist) in das Weltkoordinatensystem des Roboters zu überführen.

Daten-Output Das Ergebnis dieser Pipeline ist eine Liste von Zielkoordinaten (x, y, z) und Orientierungen (Yaw). Diese Daten werden an die Bewegungssteuerung übergeben, welche die weiteren Trajektorien für den Pick-and-Place-Vorgang generiert.

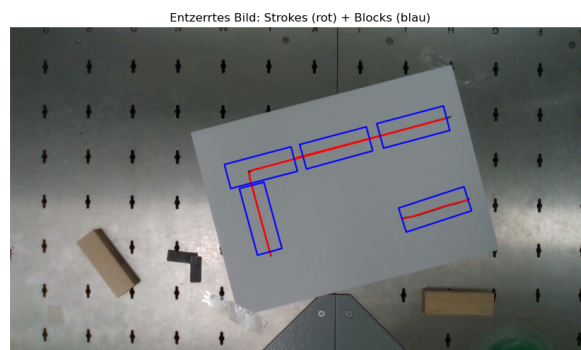


Bild 2.8. Finale Berechnung der Blockplatzierungen in Weltkoordinaten.

3 Systemablauf - Komponentenbeschreibungen

3.1 Lösungskonzept und Systemablauf

Das Kernziel der Anwendung ist die automatisierte, bildgestützte Handhabung von Bausteinen innerhalb einer Roboterzelle. Die Aufgabenstellung sieht vor, dass willkürlich platzierte, einlagig liegende Klötze erkannt und geordnet auf einer willkürlichen Linie (auf einem weißen Blatt Papier) abgelegt werden. Der Prozess wird fortgesetzt, bis entweder der verfügbare Platz auf der Ziellinie erschöpft ist oder keine weiteren Klötze im Arbeitsraum detektiert werden.

Zur Bewältigung dieser Aufgabe wird die Softwarearchitektur in zwei wesentliche Funktionsphasen unterteilt:

Phase 1: Exploration (Arbeitsraumerfassung)

In dieser Phase erfolgt die Erfassung des Arbeitsraums. Der Roboter fährt eine Sequenz vordefinierter Übersichtsposen ab, die den gesamten Arbeitsraum abdecken. Bei Erreichen jeder Pose wird ein Kamerabild aufgenommen und mittels Computer Vision sowie KI-gestützter Segmentierung (YOLO) verarbeitet, um zwei zentrale Informationen zu extrahieren:

- **Objektdetektion:** Sind Klötze im untersuchten Arbeitsraum vorzufinden? Werden Klötze erkannt, speichert das System die aktuelle Roboterpose als potenziellen Startpunkt für spätere Greifvorgänge (*Pick-Prozess*).
- **Linienextraktion:** Sind Ablagemarkierungen (Linien auf Blattpapier) im Bild enthalten? Wird eine Ablagelinie auf dem Papier identifiziert, berechnet ein Algorithmus die Ablageposen für die Bausteine entlang der Linien.

Phase 2: Pick-and-Place (Aufgabenausführung)

Nach Abschluss der Exploration wechselt das System in die aktive Ausführung der Aufgabe. Dabei folgt der Roboter einem iterativen Such- und Greifprozess:

1. **Grob-Lokalisierung:** Der Roboter fährt die erste gespeicherte Übersichtspose an. Anhand der Bilddaten wird ein Klotz ausgewählt und dessen Weltkoordinaten grob berechnet.
2. **Fein-Lokalisierung:** Zur Erhöhung der Greifpräzision positioniert sich der Roboter mit der Kamera direkt über dem Zielobjekt (Hover-Pose). Eine erneute Bildauswertung verfeinert die Positions- und Rotationsdaten.
3. **Pick-Vorgang:** Der Manipulator richtet sich entsprechend der Klotzorientierung aus, fährt die Zielposition bis zum physischen Kontakt an und aktiviert den Vakuumgreifer.

4. **Transfer und Ablegen:** Nach dem Rückzug auf eine sichere Arbeitshöhe wird die berechnete Ablageposition angefahren. Der Klotz wird entsprechend der Linienausrichtung orientiert, auf der Oberfläche abgesetzt und durch Deaktivieren des Vakuums freigegeben.
5. **Prozess-Iteration:** Der Roboter kehrt zur Übersichtspose zurück, um den nächsten Klotz zu bearbeiten, sofern die Abbruchbedingungen noch nicht erfüllt sind.

3.2 Exploration Navigator

Nutzen: Dieser Block steuert die erste Phase des Projekts (Exploration), verwaltet den Kamera-Handshake für die Bildaufnahme und fungiert in der zweiten Phase als passives Gateway für den `PickPlaceController`.

Schnittstellen

Inputs:

- `ist_pose` (EncodedState): Aktuelle Roboterpose.
- `target_pose_in` (EncodedState): Zielpose vom `PickPlaceController` (PPL).
- `take_img_in` (Bool): Kamera-Trigger-Anforderung vom PPL.
- `img_taken` (Bool): Rückmeldung der Kamera, dass ein Bild gespeichert wurde.

Outputs:

- `target_pose_out` (EncodedState): Aktueller Bewegungsbefehl an den Roboter.
- `take_img_out` (Bool): Aktiver Trigger an die Kamera.
- `trajectory_success` (Bool): Statusmeldung, ob das Ziel erreicht wurde.
- `trigger_ppl` (Bool): Signal zum Start der Phase 2 (Pick & Place).

Initialisierung (`on_configure_callback`)

- Posen für die Exploration werden aus einer YAML-Datei in eine interne Liste `_exploration_pose_list` übernommen.
- **Status-Reset:** Der interne `_state` wird auf `EXPLORATION` gesetzt.
- Die Steuersignale `_waiting_for_camera_reset`, `_trajectory_success` und `_trigger_ppl` werden explizit auf `False` initialisiert.

Detaillierte interne Logik (`on_step_callback`)

Der Block unterscheidet in jedem Taktzyklus zwischen zwei Hauptzuständen:

1. Zustand: EXPLORATION

- **Listenprüfung:** Es wird geprüft, ob die `_exploration_pose_list` noch Einträge enthält.
- **Wenn die Liste nicht leer ist:**
 - Die erste Pose der Liste (`[0]`) wird permanent an den Output `target_pose_out` gesendet.
 - **Distanzberechnung:** Der Block berechnet die euklidische Distanz zwischen der aktuellen `ist_pose` und der `target_pose_out`.
 - **Zielerreichung:** Ist die Distanz $< 0,002\text{ m}$ (2 mm) und der Block befindet sich nicht im Reset-Warten (`not _waiting_for_camera_reset`), werden `_trajectory_success` und `_take_img_out` auf `True` gesetzt.
 - **Kamera-Handshake:** Sobald der Input `img_taken` auf `True` springt, wird die aktuelle Pose mittels `pop(0)` aus der Liste gelöscht.
 - Die Signale `_take_img_out` und `_trajectory_success` werden auf `False` gesetzt und `_waiting_for_camera_reset` wird auf `True` gesetzt (Sicherheits-Lock).
 - **Reset-Logik:** Erst wenn `img_taken` wieder auf `False` abfällt, wird `_waiting_for_camera_reset` auf `False` gesetzt. Dies macht den Weg für die nächste Pose frei und verhindert ein mehrfaches Löschen von Einträgen aus der `_exploration_pose_list`.
- **Wenn `len(_exploration_pose_list) == 0`:**
 - Ein interner Timer (`_phase_done_timer`) startet.
 - Um Race-Conditions beim Wechsel von Phase 1 zu Phase 2 zu verhindern, wird erst nach Ablauf von 0,5 Sekunden `_trigger_ppl` auf `True` gesetzt und der `_state` wechselt permanent zu "GATEWAY".

2. Zustand: GATEWAY

- **Pass-Through:** Der Block schreibt in jedem Takt die Eingänge `target_pose_in` und `take_img_in` ohne Modifikation direkt auf die entsprechenden Ausgänge.
- **Aktive Überwachung:** Der Block berechnet weiterhin kontinuierlich die Distanz zwischen `ist_pose` und der vom PPL gesendeten Zielpose, um das Signal `_trajectory_success` ($< 2\text{ mm}$) weiterhin bereitzustellen.

3.3 JTC Command Generator

Nutzen: Da der Joint Trajectory Controller (JTC) asynchrone Services ausführt, muss dieser Service korrekt aufgerufen werden. Dieser Block erstellt den passenden Befehl, um den JTC aufzufordern, in einer vorgegebenen Zeit die aktuelle Pose aus dem Topic `target_pose_out` anzufahren.

Zusammenspiel mit dem AICA "Signal-to-Frame" Block

Damit der JTC den Bewegungsbefehl ausführen kann, muss dieser bei Erhalt des Befehls das anzufahrende Frame im TF-Tree finden. Der AICA-Studio-Block „Signal to Frame“ hört wie der JTC Command Generator auf das Topic `target_pose_out` und schreibt die dort veröffentlichte Pose als Frame in den TF-Tree, damit das Ziel im System bekannt ist.

Schnittstellen

Inputs:

- `ist_pose` & `target_pose(EncodedState)`: Aktuelle Ist-Position und Ziel-Pose zur Distanzberechnung.

Service-Schnittstelle (Output):

- `jtc_client (StringTrigger)`: Asynchroner Service-Call, der den berechneten Befehl als `payload` an die Robotersteuerung sendet.

Initialisierung (`on_configure_callback`)

- Erstellung des ROS 2 Service-Clients für den Typ `StringTrigger` unter dem konfigurierten `service_name`.
- Zurücksetzen des Befehlsspeichers (`_pending_command`) und der letzten Zielposition (`_last_target_pos`).

Detaillierte interne Logik (`on_step_callback`)

- **Retry-Mechanismus:** Falls ein Befehl im vorherigen Schritt aufgrund eines nicht-erreichbaren Services nicht gesendet werden konnte (`_pending_command`), wird der Sendeversuch wiederholt.
- **Flankenauswertung:** Der Block vergleicht die aktuelle `target_pose` mit der `_last_target_pos`. Nur wenn die euklidische Distanz $> 1\text{ mm}$ ($0,001\text{ m}$) ist, wird ein neuer Befehl erstellt und als entsprechender Service-Call gesendet.
- **Zeitberechnung:** Der Befehl muss die Dauer bis zum Erreichen des nächsten Frames enthalten. Dafür wird die euklidische Distanz zwischen der `ist_pose` und `target_pose` durch eine einstellbare Verfahrensgeschwindigkeit geteilt.

- **Befehls-Erstellung:** Erstellung eines Strings im YAML-Format, der den Ziel-Frame-Namen und die berechnete Dauer enthält (z. B. `{frames: [target_frame], durations: [2.5]}`).
- **Service-Call:** Absenden des Requests an den JTC-Service.

3.4 Pose Triggered Camera

Nutzen: Dieser Block soll auf Anforderung beim Erreichen einer Zielpose ein Bild aus dem Kamerastream speichern und parallel die dazugehörige Roboterposition abspeichern.

Schnittstellen

Inputs:

- `take_img` (Bool): Trigger-Anforderung für eine Bildaufnahme.
- `trajectory_success` (Bool): Statusmeldung über den Stillstand des Roboters am Ziel.
- `ist_pose_in` (EncodedState): Kontinuierliche TCP-Pose vom Roboter.
- `image_stream` (RosImage): Live-Videostream der Kamera.

Outputs:

- `img_taken` (Bool): Handshake-Signal, das den Abschluss der Aufnahme bestätigt.
- `image_out` (RosImage): Das eingefrorene Standbild (Snapshot).
- `ist_pose_out` (EncodedState): Die zum Bildzeitpunkt zugehörige TCP-Pose.
- `cam_ist_pose_out` (EncodedState): Die berechnete Kamerapose im Weltkoordinatensystem.

Detaillierte interne Logik (`on_step_callback`)

- **Trigger-Bedingung:** Der Block wartet darauf, dass der Bildwunsch `take_img` und die Ankunft am Ziel `trajectory_success` gleichzeitig `True` sind.
- **Aufnahme-Delay:** Bei erfüllter Bedingung startet ein Timer (`_is_delaying`) von 0,3 s, um den Stillstand sicherzustellen (`trajectory_success` wird bereits 2 mm vor dem Ziel `True`, zudem kann der Arm durch das Bremsen nachschwingen).
- **Bildaufnahme:** Nach Ablauf des Delays werden die Daten (Bild und Pose) gleichzeitig auf die Ausgänge geschrieben.
- **Kamerapose-Berechnung:** Für die Bildauswertung ist die Sensorposition und -ausrichtung grundlegend. Durch die Transformation von TCP zu Kamera kann anhand der TCP-Pose die Kamerapose in Weltkoordinaten beschrieben und über den Output `cam_ist_pose_out` ausgegeben werden.

- **Handshake:** Damit der Exploration Navigator nach der Bildaufnahme zur nächsten Pose weiterfährt, wird der Output `img_taken` auf `True` gesetzt und erst wieder zurückgesetzt, wenn der Roboter die aktuelle Position verlässt (`trajectory_success == False`).

3.5 YOLO Object Detector

Nutzen: Dieser Block führt die Inferenz des YOLOv11-Seg Modells durch. Er wandelt Bilddaten in 2D-Pixelkoordinaten von Bausteinen um.

Threading-Architektur

Da PyTorch (YOLO) nicht thread-safe ist, nutzt der Block eine strikte Trennung:

- **`__on_new_image` (Subscriber-Thread):** Kopiert das ankommende Bild und die Posen in einen Zwischenpuffer (`_pending_image_array`).
- **`on_step_callback` (Haupt-Thread):** Führt die eigentliche YOLO-Inferenz im Takt des Nodes aus.

Schnittstellen

Inputs:

- `image_in` (RosImage): Eingangs-Snapshot von der PoseTriggeredCamera.
- `ist_pose_in` (EncodedState): TCP-Pose zum Zeitpunkt der Bildaufnahme.
- `cam_ist_pose_in` (EncodedState): Kamerapose zum Zeitpunkt der Bildaufnahme.

Outputs:

- `yolo_corners_list` (Float64MultiArray): Flaches Array der 2D-Eckpunkte aller erkannten Klötze (Stride 8 pro Objekt).
- `ist_pose_out` / `cam_ist_pose_out`: Weiterleitung der eingefrorenen Posen zur synchronen Verarbeitung im MLM.
- `yolo_done_trigger` (Bool): Signalisiert den Abschluss der Inferenz für diesen Frame.
- `debug_image` (RosImage): In RViz einsehbare Topic, zeigt verarbeitetes Bild mit maskierten Klötzen.

Detaillierte interne Logik (on_step_callback)

- **Inferenz:** Das Bild wird in das BGR-Format konvertiert und die Inferenz durch das YOLO-Modell durchgeführt.
- **Geometrie-Ableitung:** Aus den Segmentierungsmasken wird mittels `cv2.minAreaRect` eine rotierte Bounding Box berechnet.
- **Rand-Filter:** Klötze, deren Eckpunkte weniger als 5 Pixel vom Bildrand entfernt sind, werden verworfen, um unvollständige Erkennungen zu vermeiden.
- **Verzögerter Trigger:** Das Signal `yolo_done_trigger` wird einen Taktzyklus nach den Koordinaten (`yolo_corners_list`) gesetzt, um Race-Conditions im nachfolgenden `MasterListManager` zu verhindern.

3.6 Dropoff Line Extractor

Nutzen: Dieser Block ist während der Exploration (Phase 1) aktiv. Weiße Blätter und die darauf gezeichneten Linien werden mittels Computer Vision erkannt und Ablageposen für die Klötze berechnet.

Schnittstellen

Inputs:

- `image_in` (RosImage): Eingangs-Snapshot von der Kamera.
- `cam_ist_pose` (EncodedState): Kamerapose zum Zeitpunkt der Bildaufnahme.
- `yolo_done_trigger` (Bool): Startsignal für die Linienerkennung.
- `trigger_ppl` (Bool): Signal für den Phasenwechsel zur Deaktivierung des Blocks.

Outputs:

- `line_ex_list` (Float64MultiArray): Liste der berechneten 3D-Ablageposen (Stride 7: [X, Y, Z, Qx, Qy, Qz, Qw]).

Detaillierte interne Logik (on_step_callback)

- **Inaktiv in Phase 2:** Sobald `trigger_ppl` aktiv ist, verriegelt ein `Latch` den Block, um CPU-Ressourcen zu sparen.
- **Weltkoordinaten-Pipeline:** Der Block transformiert die erkannten Linien-Pixel unter Berücksichtigung der Kamerapose in Weltkoordinaten.
- **Kollisionsgeprüfte Platzierung:** Über die Funktion `place_blocks_global` wird die Linie mit virtuellen Bausteinen „aufgefüllt“. Dabei wird sichergestellt, dass die Bausteine den definierten Abstand (`step_mm`) einhalten.

- **Output:** Die resultierenden Posen werden als `line_ex_list` (Stride 7) für den `MasterListManager` bereitgestellt.

3.7 Master List Manager

Nutzen: Der MLM-Block dient als Zwischenspeicher, um Daten aus der Exploration in der zweiten Phase zu verwenden. Daten aus dem YOLO-Block werden hier weiterverarbeitet und mit Daten des `DropoffLineExtractor`-Blocks verglichen, damit keine Klötze von der Linie gepickt werden.

Schnittstellen

Inputs:

- `yolo_done_trigger` (Bool): Polling-Trigger für neue Objektdaten.
- `yolo_corners_list_in` (Float64MultiArray): 2D-Koordinaten erkannter Klötze.
- `line_ex_list_in` (Float64MultiArray): 3D-Ablageposen vom `DropoffLineExtractor`.
- `ist_pose_in` / `cam_ist_pose_in` (EncodedState): Roboter- und Kameraposen zum Bildzeitpunkt.
- `trigger_ppl` (Bool): Steuert die Befüllung der `master_overview`.

Outputs:

- `master_overview` (Float64MultiArray): Liste der TCP-Posen aller Bildaufnahmen aus Phase 1.
- `master_dropoff` (Float64MultiArray): Persistente Liste aller gültigen Ablageorte.
- `filtered_yolo` (Float64MultiArray): Bereinigte Liste greifbarer Klötze (Stride 9: [X, Y, Area, u, v, Qx, Qy, Qz, Qw]).
- `mlm_done_trigger` (Bool): Stafetten-Trigger für den `PickPlaceController`.

Detaillierte interne Logik (on_step_callback)

- **Übersichtsposen speichern:** Während der Exploration werden alle Posen gespeichert, in denen Klötze erkannt wurden, um später von diesen Positionen aus den Pick-Prozess zu starten. Eine Pose wird nur dann in die `master_overview`-Liste aufgenommen, wenn sich das System in Phase 1 (`trigger_ppl == False`) befindet und die YOLO-Inferenz mindestens einen Klotz erkennt (`len(corners) > 0`).
- **Klotzkoordinaten:** Mittels `pinhole_ray` und `ray_table_intersect` berechnet der Block aus den 2D-Pixeln und der Tischhöhe (`_z_table`) die exakten Weltkoordinaten (X, Y) der Klötze.
- **Twist-Minimierung:** Über die Funktion `minimize_twist` wird die optimale Greif-Rotation berechnet, um die Gelenkbelastung des Roboters zu minimieren.

- **Filterung:** Klötze, die bereits innerhalb eines Radius von 1 cm um eine bekannte Ablageposition liegen, werden aus der `filtered_yolo`-Liste entfernt, um Fehlpicks bereits abgelegter Steine zu vermeiden.

3.8 Pick Place Controller

Nutzen: Dieser Block steuert bildgestützt den Pick-&-Place-Prozess anhand der zuvor durchgeführten Exploration.

(on_configure_callback)

- Der Zustand (`_state`) wird auf INIT gesetzt.
- Damit im Pick-&-Place-Prozess mit den Masterlisten des MLM gearbeitet werden kann, müssen diese im PPC als lokale Listen vorliegen. Diese werden initial geleert.

Schnittstellen

Inputs:

- `trigger_pp1` (Bool): Signalisiert das Ende der Exploration und den Start von Phase 2.
- `trajectory_success` (Bool): Rückmeldung des Roboters über das Erreichen einer Zielpose.
- `mlm_done_trigger` (Bool): Stafetten-Trigger vom Master-List-Manager, der signalisiert, dass die Daten in `filtered_yolo` aktualisiert wurden.
- `filtered_yolo` (Float64MultiArray): Live-Liste erkannter Klötze (Stride 9: Position, Fläche, Orientierung).
- `master_dropoff` / `master_overview`: Global bereitgestellte Listen für Ablage- und Suchposen.

Outputs:

- `target_pose_out` (EncodedState): Aktuelle Zielpose für den Roboter.
- `take_img_out` (Bool): Trigger, um ein Bild bei der nächsten Zielpose aufzunehmen und eine aktuelle `filtered_yolo`-Liste zu erhalten.
- `vacuum_on` (Bool): Steuersignal für den Vakuum-Greifer.

Detaillierte interne Logik (on_step_callback)

Der Controller nutzt eine State-machine zur Steuerung des Ablaufs.

- **Zustand INIT:** Wartet im Standby auf `trigger_ppl == True`. Kopiert dann einmalig die Masterlisten-Topics in lokale, veränderbare Listen und wechselt zu `CHECK_LISTS`.
- **Zustand CHECK_LISTS:** Prüft, ob noch Klötze und freie Ablageposen verfügbar sind. Wenn nicht, fährt der Roboter in die `HOME`-Position und der Block wechselt zu `FINISHED`.
- **Zustand MOVE_OVERVIEW:** Sendet die erste Übersichtspose. Bei Ankunft (`traj_rising`) wird der Bildtrigger gesetzt (`take_img_out = True`) und in `WAIT_IMG_1` auf die Bildauswertung gewartet.
- **Zustand WAIT_IMG_1:** Sobald `mlm_rising` erkannt wird, wird die neue `filtered_yolo`-Liste ausgewertet. Wurden keine Klötze gefunden, wird die aktuelle Übersichtspose verworfen und zu `CHECK_LISTS` gewechselt. Andernfalls wird der Klotz mit der größten Fläche gewählt, eine Hover-Pose berechnet (Kamera senkrecht in sicherer Arbeitshöhe über dem Klotz) und zu `MOVE_PICK_HOVER` gewechselt.
- **Zustand MOVE_PICK_HOVER & WAIT_IMG_2:** Sobald die Kamera senkrecht über dem Klotz zum Stehen kommt, wird `take_img_out = True` gesetzt und auf ein neues `filtered_yolo`-Signal gewartet. Dieses enthält erneut Klotzkoordinaten, jedoch technisch bedingt mit geringerem Fehler der X- und Y-Koordinate. Es wird eine Pick-Pose mit einer parametrisierten Z-Höhe und der im MLM berechneten Rotation des Klotzes erstellt, sowie die darauf folgende Retract-Pose.
- **Zustand EXECUTE_PICK:** Unterteilt in Sub-Zustände:
 - `APPROACH_PICK`: Anfahrt der Greifposition. Bei Ankunft wird das Vakuum aktiviert und 0,3s gewartet, damit der Klotz sicher angesaugt wird.
 - `VACUUM_DELAY`: Nach Ablauf des Timers wird die Retract-Pose gesendet.
 - `APPROACH_RETRACT`: Wartet auf das Erreichen der Retract-Pose und wechselt zu `PREPARE_PLACE`.
- **Zustand PREPARE_PLACE:** Liest die nächste Ablageposition aus der `_master_dropoff_local`-Liste, berechnet eine Hover-Pose (Ablagepose + Z-Hover-Aufschlag für sicheren Transport) und sendet diese.
- **Zustand EXECUTE_PLACE:**
 - `APPROACH_DROP`: Sendet die tatsächliche Ablagepose aus der `_master_dropoff_local`-Liste. Bei `trajectory_success == True` wird das Vakuum abgeschaltet und ein 0,5s Timer gestartet, um die sichere Ablage zu gewährleisten.
 - `RELEASE_DELAY`: Nach Ablauf der Zeit wird die Retract-Pose (gleich der Ablage-Hover-Pose) gesendet und die genutzte Ablageposition aus der Liste gelöscht.

3.9 Architektur 2: Vision Processor Komponente

Motivation und Konzept

Das Ziel dieser Architekturvariante ist die Vereinfachung der Systemstruktur sowie die Reduktion von Handshakes und der damit verbundenen Fehleranfälligkeit. Durch die Zusammenführung der Komponenten sollen Race-Conditions vermieden und die ROS/AICA-Werkzeuge effizienter nutzen. Der wesentliche Fokus liegt dabei auf der Senkung der CPU-Auslastung, da sich die Hardware-Ressourcen während der Projektumsetzung als limitierender Faktor herausstellten.

Während die grundlegende Ablauflogik unverändert bleibt, werden die bisherigen Einzelblöcke `pose_triggered_camera`, `yolo_object_detector`, `dropoff_line_extractor` und `master_list_manager` in der neuen Komponente `vision_processor` (VP) vereint. Dieser Block bedient alle übrigen Programmmodule mit den notwendigen Triggern und Daten. Da der VP analog zu den ursprünglichen Kamera- und MLM-Blöcken mit dem `ExplorationNavigator` und dem `PickPlaceController` (PPC) kommuniziert, fungiert er als abwärtskompatibles Add-on.

Daraus ergeben sich folgende **Vorteile**:

- Sequenzieller Systemablauf zur effektiven Vermeidung von Race-Conditions.
- Signifikant vereinfachte Architektur innerhalb von AICA-Studio.
- Da die Skripte und Funktionen nun intern innerhalb eines Blocks ablaufen, entfallen komplexe Handshakes sowie zahlreiche ROS-Topics, da Informationen direkt im Speicher gehalten werden. Dies führt zu einer massiven Reduktion des Signalaufkommens und der Hardwareauslastung.
- Verringerung der Systemkomplexität bei vergleichsweise geringem Implementierungsaufwand.

Der Vision Processor (VP)

Der `vision_processor` integriert den Code der ersetzten Blöcke in modifizierter Form, sodass die Datenhandhabung nun ohne externe Kommunikationswege direkt intern erfolgt.

Schnittstellen (Inputs & Outputs)

Inputs:

- `take_img` (Bool): Trigger-Signal vom `ExplorationNavigator` oder `PickPlaceController`.
- `trajectory_success` (Bool): Bestätigung, dass der Roboter die Zielpose (innerhalb einer Toleranz von 2 mm) erreicht hat.
- `ist_pose_in` (EncodedState): Aktuelle TCP-Pose des Roboters in Weltkoordinaten.
- `image_stream` (RosImage): Kontinuierlicher Bilddatenstrom der Kamera.

- `trigger_ppl` (Bool): Phase-2-Signal vom `ExplorationNavigator`.

Outputs:

- `master_overview` (Float64MultiArray): TCP-Posen aller Kameraaufnahmen aus Phase 1 (Stride 7). Die Befüllung erfolgt nur, solange `trigger_ppl` auf `False` steht.
- `master_dropoff` (Float64MultiArray): Berechnete 3D-Ablageposen entlang der detektierten Linien (Stride 7).
- `filtered_yolo` (Float64MultiArray): 3D-Greifposen aller erkannten Klötze, die sich nicht auf Ablagepositionen befinden (Stride 9).
- `vision_processor_done` (Bool): Signalisiert dem PPC, dass alle Outputs konsistent aktualisiert wurden.
- `img_taken` (Bool): Handshake-Signal an den `ExplorationNavigator`.
- `debug_image` (RosImage): RGB-Bild mit visualisierten YOLO-Boxen, Mittelpunkten und Text-Overlay zur Prozessüberwachung.

Initialisierung und interne Logik

Beim Initialisieren der Komponente in AICA-Studio wird über die Funktion `on_configure_callback` das YOLOv11-Modell geladen. Der entsprechende Dateipfad ist als anpassbarer Parameter hinterlegt.

Zu Beginn jedes Taktes (`on_step_callback`) prüft der VP das Signal `trigger_ppl`. Sobald dieses den Wert `True` annimmt, wird die Grundlinienerkennung dauerhaft deaktiviert, da diese Information nach der Explorationsphase redundant ist. Die Kommunikation mit dem PPC erfolgt über das Signal `vision_processor_done`, welches den ursprünglichen `mlm_done_trigger` ersetzt. Dieses Signal wird für genau einen Takt gehalten, sobald die Koordinatenberechnung abgeschlossen ist. Das Handshake-Signal `img_taken` wird zurückgesetzt, sobald `trajectory_success` auf `False` wechselt, was den Roboter zur Fortsetzung der Bewegung autorisiert.

Der VP verarbeitet den Bildstream direkt. Analog zum früheren `pose_triggered_camera`-Block wird bei einem Trigger eine Wartezeit von 0,3 Sekunden initiiert, um mechanische Schwingungen zu minimieren, bevor ein Frame sowie die aktuelle Ist-Pose für die Pipeline fixiert werden. Es folgen die YOLO-Inferenz (Resultat: Stride-8-Array der Bounding-Boxes) sowie – während der Exploration – die Grundlinienerkennung über das interne Objekt `_dle_algo`, welches direkt aus dem Skript des `dropoff_line_extractor` importiert wird. Abschließend berechnet die integrierte MLM-Logik die Weltkoordinaten, Flächen und Ausrichtungen (Quaternionen) der Objekte.

Zur visuellen Kontrolle in RViz wird am Ende jedes Zyklus ein `debug_image` bereitgestellt, welches die YOLO-Masken und berechneten Koordinaten als Overlay darstellt.

Entfernte Signale

Durch die interne Datenverarbeitung im `vision_processor` entfallen zahlreiche Signale und ROS-Topics, was zu einer signifikanten Entlastung des Gesamtsystems führt:

Signal	Von → Nach (Alt)	Zweck
<code>image_out</code>	<code>pose_triggered_camera</code> → <code>yolo_object_detector</code>	Eingefrorenes Bild als ROS-Topic
<code>ist_pose_out</code>	<code>pose_triggered_camera</code> → <code>yolo_object_detector</code>	Eingefrorene TCP-Pose
<code>cam_ist_pose_out</code>	<code>pose_triggered_camera</code> → <code>yolo_object_detector</code>	Berechnete Kamerapose
<code>ist_pose_out</code>	<code>yolo_object_detector</code> → <code>master_list_manager</code>	TCP-Pose (Durchreichung)
<code>cam_ist_pose_out</code>	<code>yolo_object_detector</code> → <code>master_list_manager</code>	Kamerapose (Durchreichung)
<code>yolo_corners_list</code>	<code>yolo_object_detector</code> → <code>dle</code> + <code>mlm</code>	Stride-8 Eckpunkte (Pixel)
<code>yolo_done_trigger</code>	<code>yolo_object_detector</code> → <code>dle</code> + <code>mlm</code>	Puls-Trigger für Folgeprozesse
<code>line_ex_list</code>	<code>dropoff_line_extractor</code> → <code>master_list_manager</code>	Stride-7 Ablageposen

Tabelle 3.1. Übersicht der entfernten Signale durch Bündelung im Vision Processor.

Den um den unterschied der Architekturen grafisch nachzuvollziehen sind im Anhang zwei Abbildungen von beider ACIA Studio Anwendungen (v1 & v2) enthalten. Die Architektur v2 ist zeichnet sich durch eine kompaktere Ansicht aus, während die Architektur v1 mehr Komponenten und Verbindungslinien enthält.

4 Probleme und Schwierigkeiten

4.1 Evaluierung und konstruktives Feedback zur AICA-Plattform

In diesem Abschnitt werden die identifizierten technischen Hürden aufgezeigt, die maßgeblichen Einfluss auf die Effizienz der Anwendungsentwicklung haben. Nutzen soll diese Sammlung Anwendern von Folgeprojekten mit AICA, als AICA selbst, um eine weitere Kundensicht kennenzulernen.

Hardware-Integration und I/O-Steuerung

Ein zentrales Hindernis war die Interaktion zwischen der AICA-Software und der Kuka-Robotersteuerung.

- **Inbetriebnahme:** Der Installationsprozess und die initiale Kommunikation zwischen Roboterzelle und AICA über RSI erwiesen sich als überdurchschnittlich komplex und konnten nur durch direkten Support von AICA etabliert werden.
- **I/O-Zugriff:** Über die AICA-Schnittstellen war kein Zugriff auf die I/O-Controller des Roboters möglich. Dies verhinderte die direkte Ansteuerung der Werkzeugperipherie (z. B. Greifer).

Ressourceneffizienz und Container-Laufzeit

Die Systemperformance stellte sich als kritischer Flaschenhals heraus:

- **CPU-Last:** Die hochfrequente Taktung der AICA-Komponenten führt zu einer immensen Rechenlast, die auf Standard-Hardware regelmäßig zu Systemüberlastungen und Kommunikationsfehlern führt.
- **GPU-Virtualisierung:** Trotz Befolgung der offiziellen Dokumentation konnte keine CUDA-Unterstützung innerhalb des Docker-Containers realisiert werden. Die notwendige Ausführung der KI-Inferenz auf der CPU verschärfte die Hardware-Limitierungen zusätzlich.
- **Synchronisationsfehler:** Inkonsistenzen bei der Übernahme von Code-Änderungen im laufenden Container zwangen das Team zu häufigen Full-Rebuilds der Pakete, was den Entwicklungszyklus massiv verlangsamte.

Funktionsumfang und Steuerungsmethoden

Für eine effiziente Applikationserstellung fehlen derzeit noch elementare Standard-Komponenten:

- **Bewegungssteuerung:** Eine AICA Komponente die Point-to-Point (P2P) Bewegungen mit dem JTC ausführt gibt es noch nicht und zwingt den Nutzer den IK Velocity Controller zu verwenden oder sich diese Komponente selber zu erstellen.
- **Kalibrierung:** Eine integrierte Lösung zur Kamerakalibrierung innerhalb von AICA-Studio würde den manuellen Setup-Aufwand erheblich reduzieren.
- **Unintuitives Lifecycle-Management:** Die Orchestrierung verschiedener Programmphasen über die nativen Lifecycle-Zustände (*Active/Inactive*) der Komponenten erwies sich als wenig intuitiv. Da automatisierte Übergänge zwischen komplexen Phasen (z. B. von der Exploration zum Pick-and-Place) nicht einfach grafisch konfigurierbar waren, wurde die Ablaufsteuerung primär über manuelle Merker (Flags) direkt im Quellcode realisiert.

Dokumentation und API-Stabilität

Die Dokumentation weist in Bezug auf fortgeschrittene Anwendungen Lücken auf:

- **Service-Adressierung:** Während der Projektlaufzeit verhielten sich Service-Adressen (z. B. `/hardware/jtc_controller`) inkonsistent, was zu schwer diagnostizierbaren Fehlern führte.
- **Spezifikationen:** Es mangelt an detaillierten Spezifikationen zu Signalformaten und der Implementierung eigener Services. Die vorliegenden Beispiele decken oft nur triviale Anwendungsfälle ab.

Sensorik-Einschränkungen

Es wurde festgestellt, dass die Integration der Intel RealSense-Kamera bei geringen Arbeitsabständen (ca. 30 cm) an ihre technischen Grenzen stößt. Die starken Schwankungen der Tiefendaten machten diese für eine präzise Steuerung unbrauchbar, weshalb das Projekt auf reine 2D-Bildverarbeitung umgestellt wurde.

5 Zusammenfassung und Fazit

5.1 Zusammenfassung

Im Rahmen dieser Arbeit wurde an der Hochschule Karlsruhe erstmals die Plattform AICA für die Entwicklung einer alternativen Robotersteuerung eingesetzt. Ziel war es, eine bildbasierte Wahrnehmungspipeline zu entwerfen, die Bausteine zuverlässig erkennt und lokalisiert. Hierfür wurden spezialisierte ROS/AICA-Komponenten entwickelt, die auf dem YOLOv11-Verfahren zur Instanzsegmentierung basieren. In der Simulation konnte ein korrektes Systemverhalten sowie eine verlässliche Erfüllung des Aufgabenzwecks nachgewiesen werden. Dabei wurden zwei verschiedene Architekturansätze gegenübergestellt, wobei der Fokus auf der Optimierung der Datenflüsse und der Reduktion der Systemkomplexität lag.

5.2 Fazit

Die Projektergebnisse zeigen, dass die entwickelten Bildverarbeitungstools die Anforderungen zuverlässig erfüllen. Dennoch ergaben sich in der Realanwendung erhebliche technische Hürden. Die hohe CPU-Last führte zu einer Instabilität des Gesamtsystems, wodurch eine Erprobung unter realen Bedingungen nur eingeschränkt möglich war. Ein kritisches Defizit stellte zudem die fehlende Ansteuerung des Vakuumgreifers über die I/O-Schnittstelle zwischen AICA und der Kuka-Steuerung dar, was einen vollständig automatisierten Pick-and-Place-Zyklus verhinderte.

Ein wesentlicher Lerneffekt ergab sich aus der Auseinandersetzung mit der ROS/AICA-Architektur. Viele der systemischen Vorteile wurden dem Projektteam erst in einer späten Projektphase vollständig deutlich, was schließlich in der Entwicklung der optimierten „Architektur 2“ mündete. Um den Einstiegsaufwand für zukünftige Projektteams zu minimieren, wurden alle Erkenntnisse und Best-Practices zur Entwicklung KI-gestützter AICA-Komponenten in einer Dokumentationsstruktur (.md-Dateien) gesammelt. Dies soll die Zeit bis zum ersten funktionsfähigen „Custom Package“ für nachfolgende Teams signifikant verkürzen.

Tabellenverzeichnis

2.1	Ergebnisse der Fehleranalyse und Inferenzzeiten der trainierten Modelle	6
3.1	Übersicht der entfernten Signale durch Bündelung im Vision Processor.	24

Abbildungsverzeichnis

2.1	Datenverteilung der Bauklötze pro Bild.	5
2.2	Annotiertes Beispielbild in der Plattform Roboflow: Kombination aus Grundlinienblatt und Blöcken.	6
2.3	Ergebnisse der Modelle bezüglich False-Positiv- und False-Negativ-Detektionen.	7
2.4	Beispiel-Testbild des optimierten YOLOv11-Nano-Modells mit Masken und Boundingboxen.	7
2.5	Kamera-Eingangsbild der Arbeitsfläche.	8
2.6	Visualisierung der Papermask-Erstellung und Blatt-Segmentierung.	9
2.7	Darstellung der extrahierten Skelett-Linien.	10
2.8	Finale Berechnung der Blockplatzierungen in Weltkoordinaten.	11
1	Anwendung in AICA Studio mit der initialen Architektur v1.	29
2	Anwendung in AICA Studio mit der zweiten Architektur v2.	29
3	Komponenten 1-3 in AICA Studio.	30
4	Komponenten 4-6 in AICA Studio.	30
5	Komponenten 7 & 8 in AICA Studio.	30

Literatur

- [1] *Annotationstool Roboflow*. <https://app.roboflow.com/>. Aufgerufen am 26.04.2026.
- [2] Bolya, D., Zhou, C., Xiao, F. ; Lee, Y. J. „YOLACT: Real-time Instance Segmentation“. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2019.
- [3] Khanam, R. ; Hussain, M. „YOLOv11: An Overview of the Key Architectural Enhancements“. In: *arXiv preprint arXiv:2410.17725* (2024).
- [4] Sapkota, R. et al. „Comparing YOLOv8 and Mask R-CNN for instance segmentation in complex orchard environments“. In: *Artificial Intelligence in Agriculture* (2024).
- [5] Ultralytics. *YOLO11 Models and Documentation*. <https://docs.ultralytics.com/models/yolo11/>. Aufgerufen am 26.04.2026.
- [6] Wang, A. et al. „YOLOv10: Real-Time End-to-End Object Detection“. In: *Advances in Neural Information Processing Systems (NeurIPS)* (2024).

Anhang

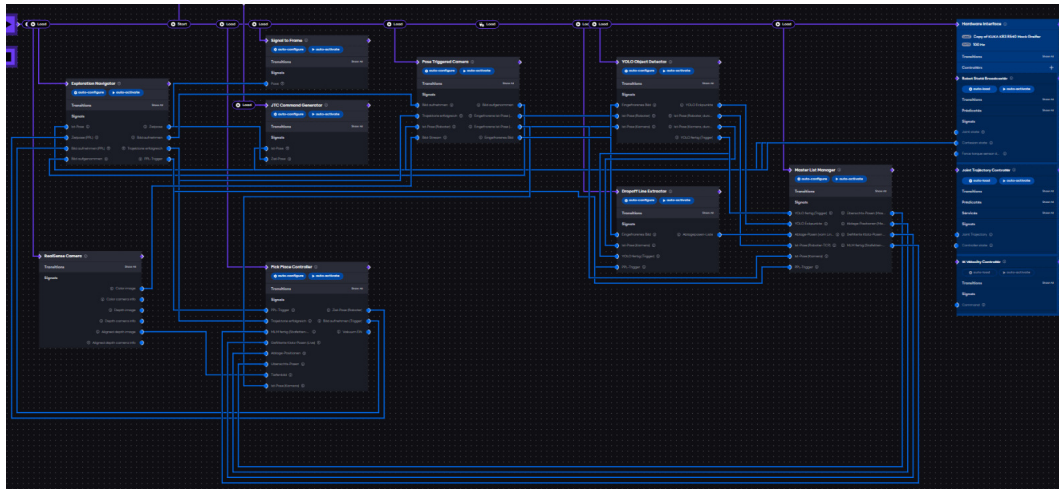


Bild 1. Anwendung in AICA Studio mit der initialen Architektur v1.

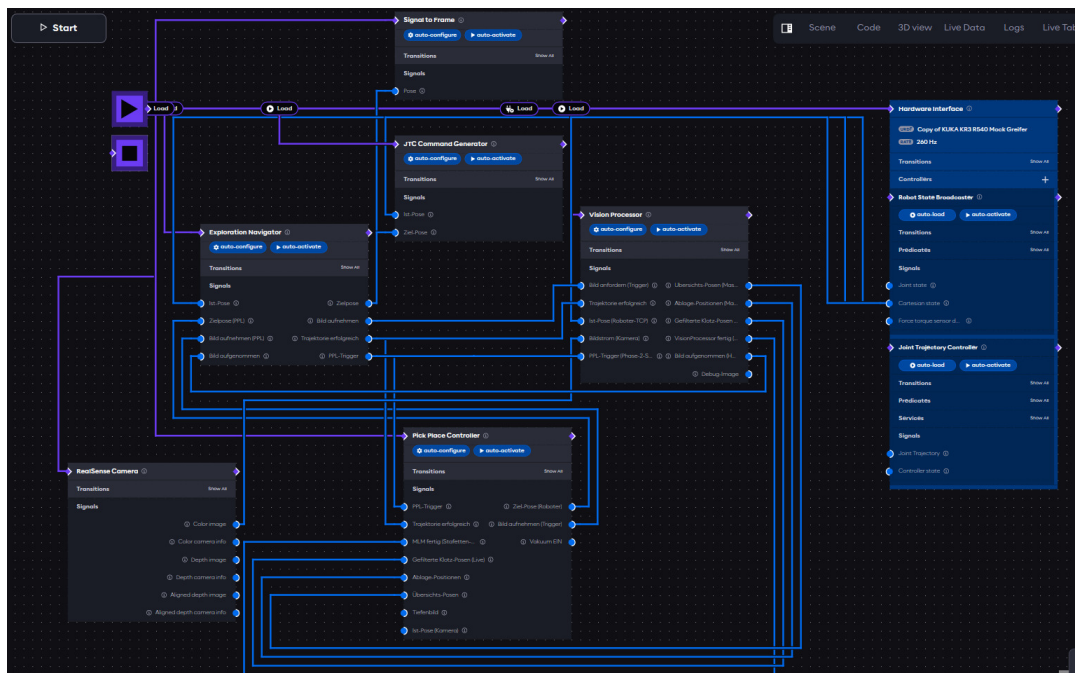
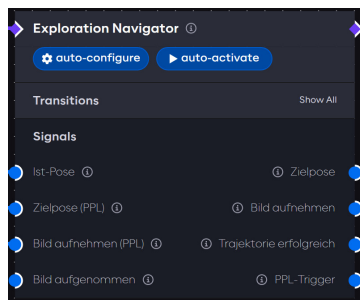
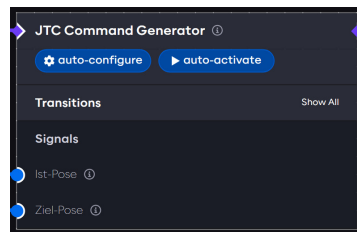


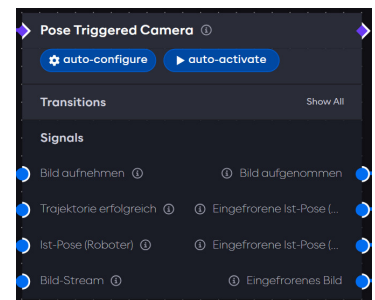
Bild 2. Anwendung in AICA Studio mit der zweiten Architektur v2.



(a) Exploration Navigator Block

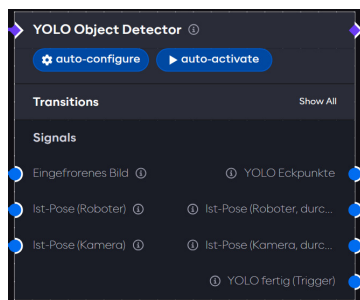


(b) JTC Command Generator

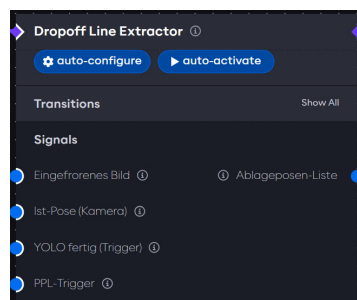


(c) Pose Triggered Camera Block

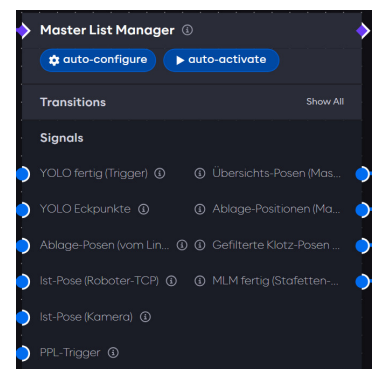
Bild 3. Komponenten 1-3 in AICA Studio.



(a) YOLO Block

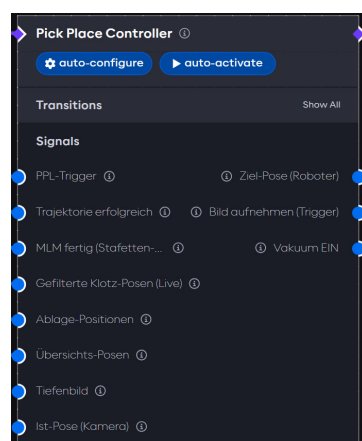


(b) Block für die Linienerkennung

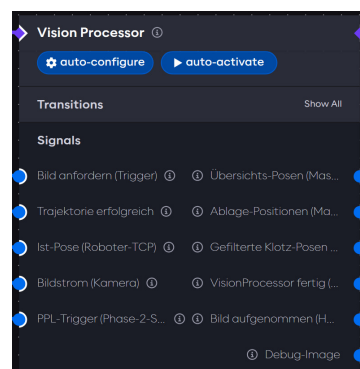


(c) MLM Block

Bild 4. Komponenten 4-6 in AICA Studio.



(a) Pick&Place-Controller Block



(b) Vision Processor Block

Bild 5. Komponenten 7 & 8 in AICA Studio.